

December 3, 2021

Shortest paths

Basic questions:

- ① Find a shortest path from a given vertex u to a given vertex v
- ② Find the shortest paths from a given vertex u to all other vertices
- ③ Find the shortest paths from u to v for all pairs (u, v) in the graph

Today topic is ②

We can avoid negative cycles if the weights are non-negative, i.e., $w : E \rightarrow \mathbb{R}^+$.

Dijkstra algorithm

Dijkstra algorithm finds shortest paths from one source vertex s , when the weights are non-negative.

The algorithm uses an array D containing the length of a shortest path from s to v we discovered till now for each vertex v of G

Dijkstra algorithm can be considered a continuous version of the breadth first search algorithm.

We assume that G is given by its adjacency matrix

Dijkstra (G, s)

$H = \{s\}$

for each $u \in V$ do

$D[u] := C[s, u]$

$\leftarrow C$ is the adj. matrix

for $i := 1$ to $|V| - 1$ do

select an $x \in V \setminus H$ such that $D[x]$ is minimal

$H := H \cup \{x\}$

$\leftarrow$ declaring that $D[x] = d(s, x)$

for each $v \in V \setminus H$ do

$D[v] := \min \{ D[v], D[x] + C[x, v] \}$

Cost : $O(|V|^2)$

Constructions of shortest paths from S :

(similar to BFS)

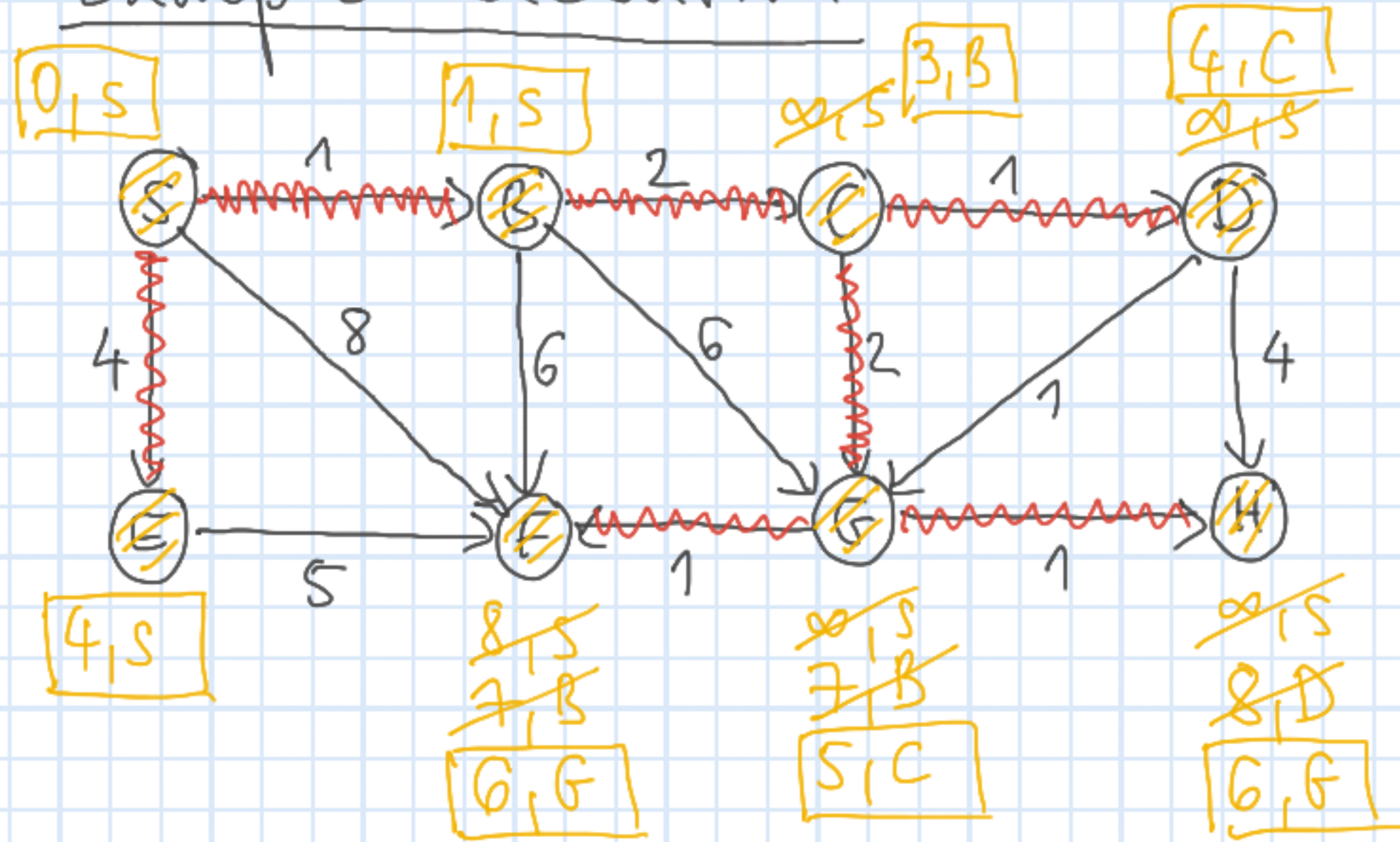
For each vertex v we define a "parent pointer".

Initially $\text{parent}[v] = S$

When we update $D[v]$ (by x) then the new parent will be x .

After termination $\text{parent}[v]$ is the last but one vertex of a shortest path from S to v , a shortest path can be obtained by following the parent pointers from v to S .

Sample execution



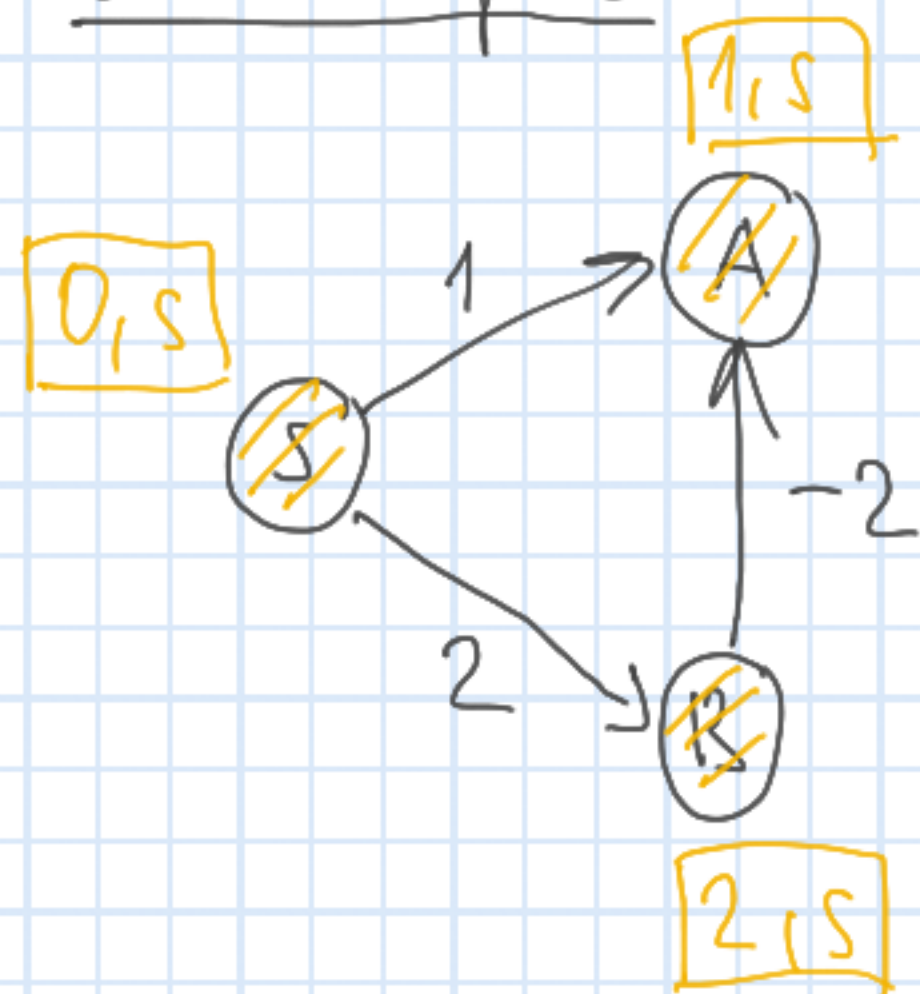
④

S, B, C, D, E, G, F, H

Shortest path tree

How important here is to rule out negative edges?

Example



Run Dijkstra!

H: S, A, B

However, the length of
 $S \rightarrow B \rightarrow A$ is $0 < 1$

Algorithms allowing the presence of negative edges (but no negative cycles, of course)

Shortest path from one source

→ Bellman-Ford algorithm

All-pairs shortest paths

→ Floyd-Warshall algorithm

Although FW solves a more general problem, the time complexities are the same $\rightarrow O(V^3)$

Both algorithms are so-called dynamic programming algorithms

→ recursive problem solving technique

→ the algorithm is not recursive

recursive : top-down

non-recursive : bottom-up

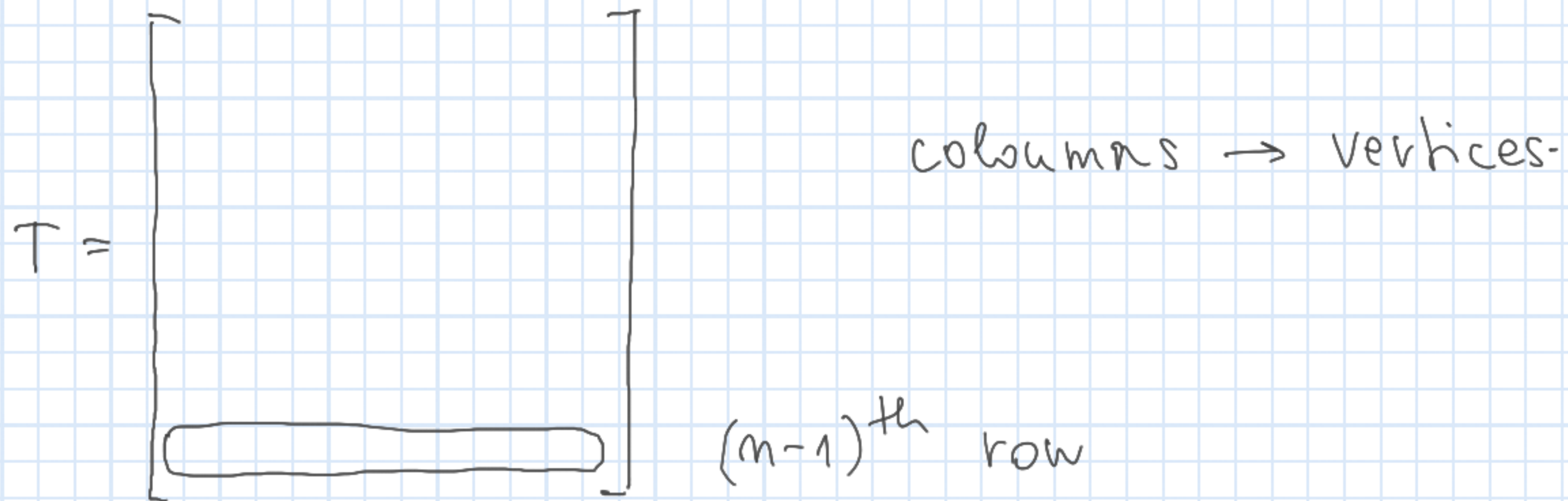
Another design technique : greedy

Prim, Kruskal, Dijkstra

Bellman - Ford algorithm

Let $V = \{1, 2, \dots, n\}$ and $s = 1$ for the sake of simplicity.

Let $T[i, j]$ denote the length of a shortest path from vertex (1) to vertex (j) with the property that the number of edges on the path is not greater than i
($1 \leq j \leq n, 1 \leq i \leq n-1$)



Since we always have a simple shortest path (without vertex repetition), $T[n-1, j]$ will contain the length of (unconditional) shortest paths from $\textcircled{1}$ to $\textcircled{j}$

Rule for calculation (recursive formula)

$$T[1, j] = C[1, j] \quad (\text{first row})$$

$$T[i, j] = \min \left\{ \begin{array}{l} T[i-1, j] \\ \min \{ T[i-1, l] + C[l, j] \mid (l, j) \in E \} \end{array} \right.$$



the part of a shortest path from ① to ①
between ① and ② is a shortest path from ① to ②

The next row can be calculated from the elements of the previous row:

Bellman - Ford (F)

for $j := 1$ to n do

$T[1, j] := C[1, j]$

for $i := 2$ to $n-1$ do

for $j := 1$ to n do

$T[i, j] := T[i-1, j]$

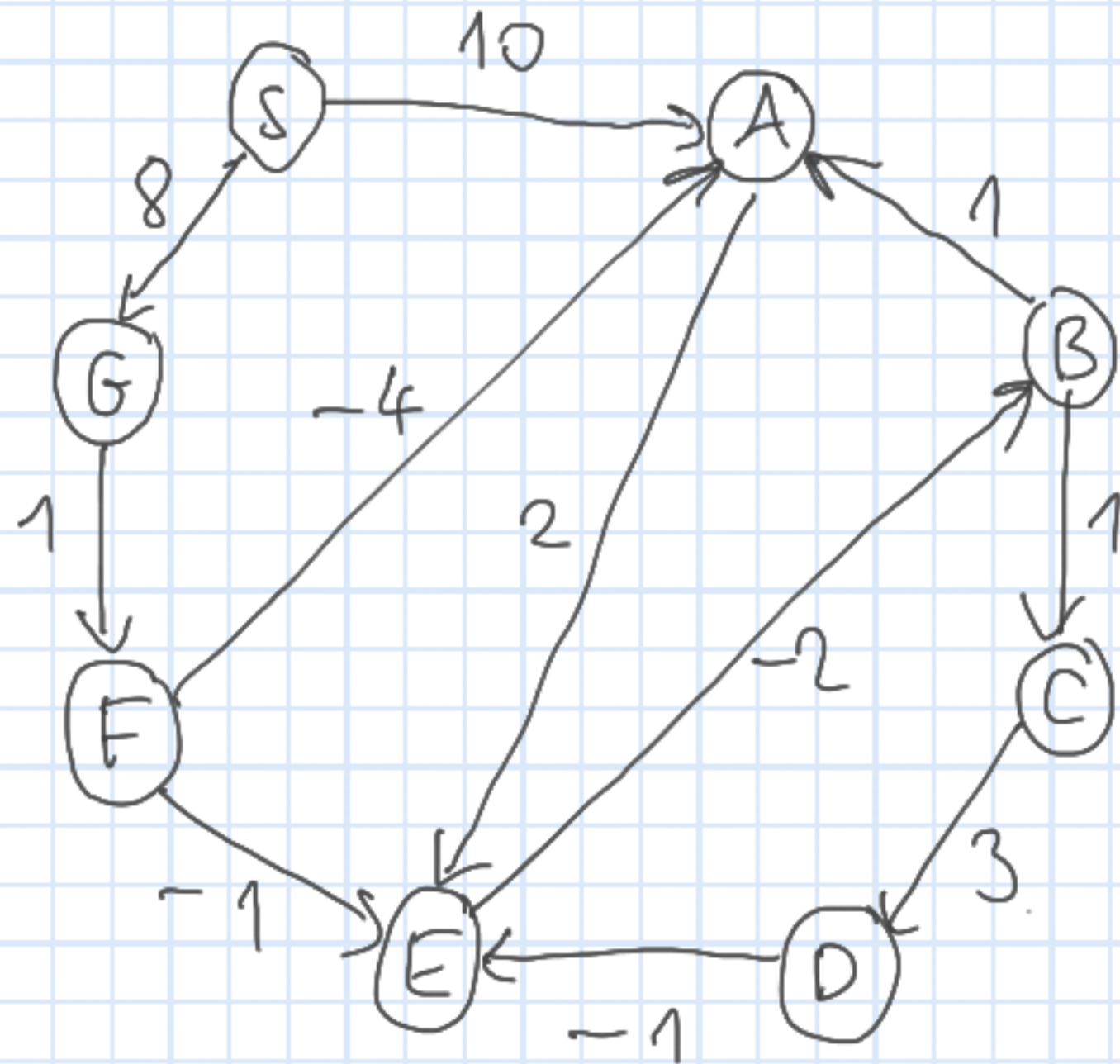
for $l := 1$ to n do

if $l \neq j$ and $T[i, j] > T[i-1, j] + C[l, j]$

then $T[i, j] := T[i-1, j] + C[l, j]$

Homework:

Execute BF alg for the next graph



i \ j	S	A	B	C	D	E	F	G
1	0	10	∞	∞	∞	∞	∞	8
2	0							
3	0							
4	0							
5	0							
6	0							
7	0							

